

Exploring the Ability of a Linear Regression Model in Predicting Housing Prices

Mathematics: Analysis and Approaches HL

Internal Assessment

Nikita Manushin

June 2026

1 Introduction

1.1 Personal Connection

When I started learning math, in the very first grade, I have always wondered how connected, the numbers and the real world are. In the past few years Neural Networks became so powerful they could communicate like humans, all of that using tons of numbers, that create words and sentences. This linear regression model I am about to present, is an example, how what seem to be meaningless numbers can make predictions about something that humans care about.

1.2 Aims of the Investigation

In this paper, I will explore how a linear regression model can be trained using gradient descent and adjusted so that it predicts current prices of apartments in Amsterdam. I will look into three features of an apartment: living area,

construction year and the distance from the center of Amsterdam. I will implement those features as independent variables into the model one by one, to see the influence of each of them on the price. The reason I am using gradient descent rather than ordinary least squares for solving my model is that I want to resemble a neural network, which is what I take my inspiration from for this model. Furthermore, this allows me to observe and explore how weights and biases are optimized iteratively using the gradients of the loss function.

1.3 Research Question

To what extent does increasing the number of input features for a linear regression model improve its accuracy at predicting housing current prices of apartments in Amsterdam.

2 Theory

2.1 A Simple Linear Regression Model

Linear regression is a tool used to predict a continuous dependent output variable based on one or more independent input variables by fitting a line, a surface or a higher-dimensional shape to minimize the loss.

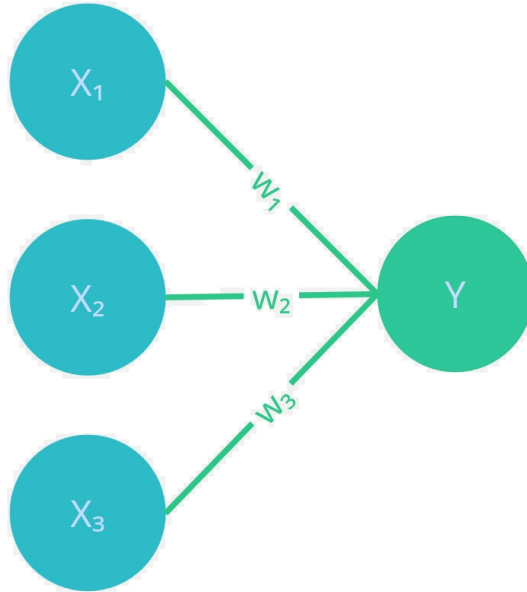


Figure 1: Model Representation

This is the structure of the linear regression model I will be using. It consists of independent variables x_1 , x_2 , x_3 called inputs and a dependent variable y called output. Inputs describe features of a particular apartment: x_1 represents the living area, x_2 is the construction year and x_3 represents the distance between the apartment and the center of Amsterdam. The output y is the price of the apartment.

A linear function that has three inputs should have three slopes, which in machine learning are called weights w_n , where w_n is the weight of x_n . Weights are constants that are used to create the linear function. However, a linear function also requires a y-intercept, which in machine learning is called a bias b . The whole function of y looks like:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + b$$

The function suggests that the price of an apartment can be calculated using those three inputs and by multiplying them by their weights and adding a bias. The weights and biases are constants.

2.2 Machine Learning and Gradient Descent

The goal of machine learning is to find the specific weights and bias that will predict the output the most accurate.

In order to calculate the accuracy of weights and biases a loss function is used. For this paper I will be using MSE (mean squared error) loss that looks like: $L(y) = \frac{1}{2}(y - t)^2$, where y is the prediction of the model and t is the target output. $(y - t)$ calculates the error the model has made, squaring removes any negative results and $\frac{1}{2}$ makes the derivative cleaner: $\partial L / \partial y = y - t$.

Machine learning is the process of finding the correct weights and biases that minimize the loss across all of the data points. Let's take a look at this diagram of a linear regression model that learns to predict housing prices in Amsterdam from the area of the apartment. The input is the area, and the output is the price of the house.

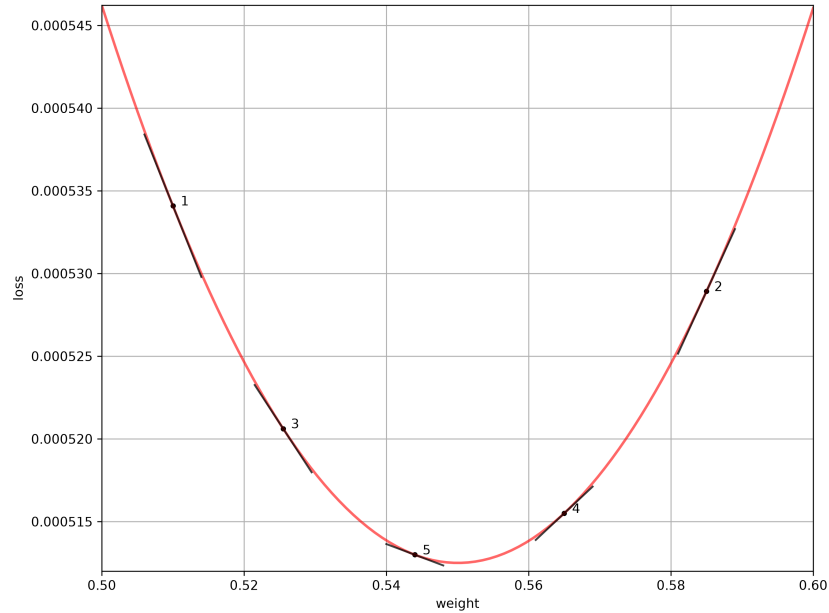


Figure 2: Gradient Descent with Fixed Bias

On the graph, the bias is fixed so that it is easier to visualize. Now, to minimize the loss, gradient descent is used. It works by finding the gradients of weight(s) and bias(es) (here it is fixed) and then moving in the direction of the steepest gradient, achieving the minimum of the function.

At the very start, the weight is random; it is around 0.51. The gradient is calculated and subtracted from the weight, because we want to minimize the loss. The same thing is repeated as many times as possible to achieve the smallest loss, and the gradient ensures that the loss is decreasing.

The gradient of weight in this graph is just the partial derivative of the loss by the weight. I will start off with writing down all of the equations we have:

$$L(y) = \frac{1}{2}(y - t)^2 \quad (1)$$

$$y = wx + b \quad (2)$$

Now we derive:

$$\partial L / \partial y = y - t \quad (3)$$

Partial derivative by w , so I treat b as a constant:

$$\partial y / \partial w = x$$

Then combine using chain rule:

$$\begin{aligned} \partial L / \partial w &= \partial L / \partial y \times \partial y / \partial w = \\ &= (y - t) \times x \end{aligned} \quad (4)$$

This is how the model knows how to change the weight. In order to minimize loss, we subtract the derivative from the weight.

$$w = w - \partial L / \partial w$$

However, the derivative might be a too big, and the weight could change too much. This could cause instability, as the weight could overshoot all the time, and it might never reach the minimum. To address this a learning rate constant is introduced - η . The derivative is multiplied by it too ensure that the steps are small so that the minimum is achieved no matter what. The formula for weight change becomes:

$$w = w - \partial L / \partial w \times \eta \quad (5)$$

η is usually somewhere around 0.1 to 0.001.

Now the same thing must be repeated with the bias, although I treated it as a constant for the sake of the clean visuals, in the process of learning the bias must also be adjusted.

The equations:

$$L = \frac{1}{2}(y - t)^2$$

$$y = wx + b$$

Derived previously:

$$\partial L / \partial y = y - t$$

Patial derivative by bias, treating weight as a constant.

$$\partial y / \partial b = 1$$

Chain rule:

$$\begin{aligned} \partial L / \partial b &= \partial L / \partial y \times \partial y / \partial b = \\ &= (y - t) \times 1 = \\ &= y - t \end{aligned} \quad (6)$$

And now the bias is changed:

$$b = b - \partial L / \partial b \times \eta \quad (7)$$

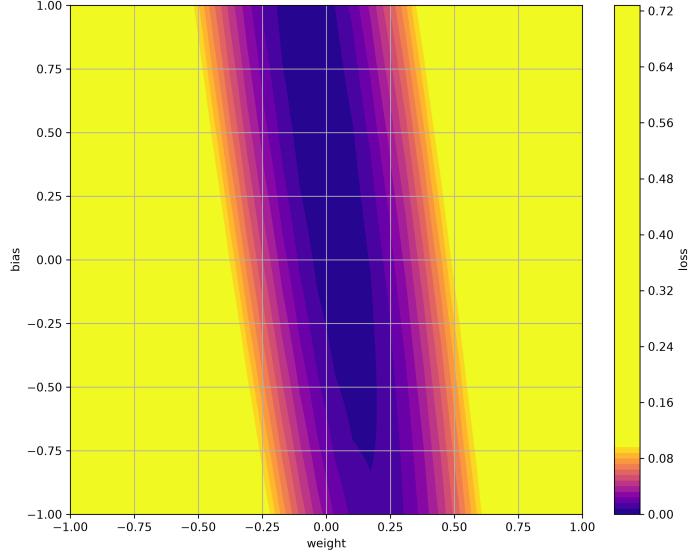


Figure 3: Countour Map of $L(w, b)$

This diagram illustrates the loss function with parameters of weight and bias, instead of fixing one of them. The x and y axis are weight and bias, respectively, and the color represents loss as shown on the right side with the colorbar.

$$L(w, b) = \frac{1}{2}((wx + b) - t)^2$$

Apart from showing the dependency of loss on weight and bias, this diagram also highlights the limitation of the learning method. Gradient decent finds the local minimum of loss function, however this particular local minimum that the model would converge to might not be the global minimum.

One of the things I have not mentioned yet is where I got the x and t from.

As I said x is the living area, and t is the price of an apartment. I have collected data from funda.nl and I only picked the apartments that are in Amsterdam and that are for sale in July of 2026. In total, I have collected around 4500 apartments with their respective prices and features.

Because the function does calculation of only one apartment at a time, I took average of all losses for each apartment with a certain weight and bias and that is what is being presented on the previous graphs. That is why I have not used x nor t as arguments for functions y and L , for the process of learning it is treated as a constant. The weight and bias are the independent variables that must be changed. After the model has finished its learning and, therefore, has constant weight and bias, the function will become $y(x) = wx + b$ and x will be the independent variable again.

2.3 Not Normalized

Another thing that must be done for a model to work is normalization of inputs and outputs. Here is an example, that will explain the importance of normalization:

I will take learning rate as 0.001, it is 10 times less than I normally would use because this will illustrate the influence of normalization clearer.

An apartment that is located at 1061 AV Amsterdam has price is 930 000 € and it has living area of 108 m^2 . Let w be 8700 and b be 348, those are values that achieve a decent amount of accuracy for this example. The function is:

$$y(x) = 8700 \times x + 348$$

x is the living area, so it is 108 m^2 . y is the predicted price:

$$y(x) = 8700 \times 108 + 348 = 939948 \text{ €}$$

t is the target price, so it is 930 000 €. Using t and y we can now calculate the loss:

$$\begin{aligned} L(x) &= \frac{1}{2}(y(x) - 930000)^2 \\ \Rightarrow L(108) &= \frac{1}{2}(939948 - 930000)^2 = \\ &= 4.95 \times 10^7 \end{aligned}$$

Now we calculate the gradient decent:

$$\begin{aligned} \partial L / \partial w &= (y - t) \times x = (939948 - 930000) \times 108 = 1074.4 \times 10^3 \\ w &= w - \partial L / \partial w \times \eta = (8700 - 1074.4 \times 10^3) \times 0.001 = 7625.616 \\ \partial L / \partial b &= (y - t) = (939948 - 930000) = 9.9 \times 10^3 \\ b &= b - \partial L / \partial b \times \eta = (348 - 9.9 \times 10^3) \times 0.001 = 338.052 \end{aligned}$$

And calculate the loss again with new weight and bias:

$$\begin{aligned} y(108) &= 7625.616 \times 108 + 338.052 = \\ &= 823904.58 \text{ €} \\ L(108) &= \frac{1}{2}(y - t)^2 = \frac{1}{2}(823904.58 - 930000)^2 = \\ &= 562.8 \times 10^7 \end{aligned}$$

The new loss is significantly larger than the first one: $4.95 \times 10^7 < 562.8 \times 10^7$. Although the gradient descent should have decreased the loss, it overshoot instead. The initial weights and biases were definitely wrong, as the first prediction was around 54 €. However, because of the huge inputs - 108 and the target outputs - 930 000, the gradients skyrocketed: 1074.4×10^3 , 9.9×10^3 . Although

the learning rate was small - 0.001, smaller than normal, the weight and bias still overshoot. This experiment can be repeated with any other weights and most of the time, the derivative will overshoot and the model will not minimize the loss function and therefore will not learn.

2.4 Normalization

To address this normalization of data is introduced. The goal is to fit any inputs and outputs into range from 0 to 1. For that this formula can be used:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (8)$$

This is done for every piece of data: both price and living area and other features. Normalization will ensure that weights are also in a small range, usually from -1 to 1, rarely might reach -2, or 2. This also allows for a larger learning rate, and 0.1 will be used here, otherwise the result will be difficult to see.

For my data the min and max for prices and living areas are respectively:

$$\begin{aligned} 189k - 9850k \text{ €} \\ 19 - 500 \text{ m}^2 \end{aligned}$$

Also, I divide the price by 10^3 to make the difference between prices more significant, and it is clearer to visualize the prices in thousand euros instead of just euros, therefore I will be using k € from now on.

I will repeat the same example but with normalized values now, the weight and biases are also chosen so that the function is somewhat accurate:

$$w = 0.5$$

$$b = -0.03$$

$$x = 108 \text{ } m^2$$

$$t = 930k \text{ } \text{€}$$

$$\begin{aligned} x_{\text{norm}} &= (x - x_{\min}) / (x_{\max} - x_{\min}) \\ &= (108 - 19) / (500 - 19) = 0.185 \\ t_{\text{norm}} &= (t - t_{\min}) / (t_{\max} - t_{\min}) \\ &= (930 - 189) / (9850 - 189) = 0.0767 \end{aligned}$$

Since all of the values will be normalized from now on, I will not use the `_norm` suffix.

$$\begin{aligned} y &= 0.5 \times x - 0.03 = 0.5 \times 0.185 - 0.03 = 0.0625 \\ L &= \frac{1}{2} (y - t)^2 = \frac{1}{2} (0.0625 - 0.0767)^2 = 1.006 \times 10^{-4} \end{aligned}$$

$$\begin{aligned} \partial L / \partial w &= (y - t) \times x = (0.0625 - 0.0767) \times 0.185 = -2.62 \times 10^{-3} \\ w &= w - \partial L / \partial w \times \eta = 0.5 - -0.00262 \times 0.1 = 0.50026 \end{aligned}$$

$$\begin{aligned} \partial L / \partial b &= (y - t) = 0.0625 - 0.0767 = -1.42 \times 10^{-2} \\ b &= b - \partial L / \partial b \times \eta = -0.03 - -0.0142 \times 0.1 = -0.0286 \end{aligned}$$

$$y = wx + b = 0.50026 \times 0.185 - 0.0286 = 0.064$$

$$L = \frac{1}{2}(y - t)^2 = \frac{1}{2}(0.064 - 0.0767)^2 = 0.809 \times 10^{-4}$$

As can be seen here the loss decreased from 1.00×10^{-4} to 0.81×10^{-4} . Normalization allows for a more stable learning process and less overshooting during the gradient descent. Moreover, normalization helps with weights and biases initialization, as we can assume that they are going to be most frequently around -1.0 to 1.0. On the other hand, without normalization the weights might go to thousands like in the last example. The diagram below shows how centered the blue graph (normalized data) to the zero compared to the red graph (not normalized data). Furthermore, the gradient is much less steep for the blue graph, which again shows that the gradient descent will be more smooth and less overshooting.

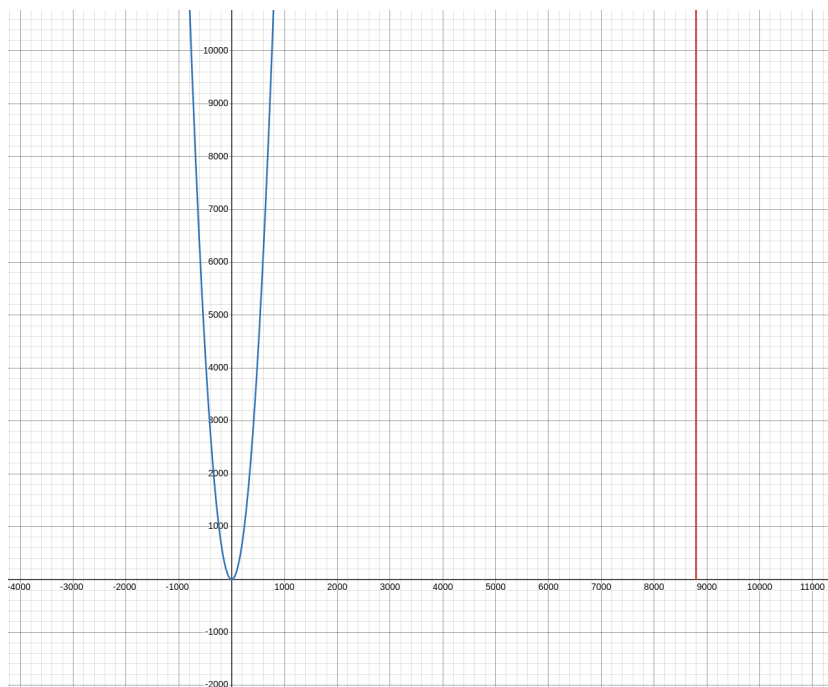


Figure 4: Normalized vs not Normalized

2.5 Learning Process

I have a few thousand apartments data from which my model must learn. The term epoch is introduced at this point, which refers to a single, complete pass of the entire training dataset through the model. In this case, gradient descent will be done for each apartment in the dataset. x is the living area, and t is the respective price for the apartments. This will make the model find the middle ground for weight and bias to minimize the loss across all the apartments. For a model to minimize loss as much as possible, numerous epochs are done. Depending on the length of the dataset this number can range, but I will be doing around 100-1000 epochs.

As I mentioned earlier, the dataset includes around 4500 entries; it is exactly 4577 apartments. I have separated the data into training and test datasets. The

training entries are used to train the model, in the process mentioned above. On the other hand, the test dataset is used to test the accuracy of predictions of the model. The test process illustrates how good the model performs on apartments it has never seen before. To separate the dataset into training and test datasets, I will be using a 0.95 ratio - 4348 and 229 entries, respectively.

Here is the model I have trained on my dataset with the structure $y = wx + b$, learning rate 0.01 and 1000 epochs. Blue points are the entries from the dataset, and the red dots are the predictions made by the model.

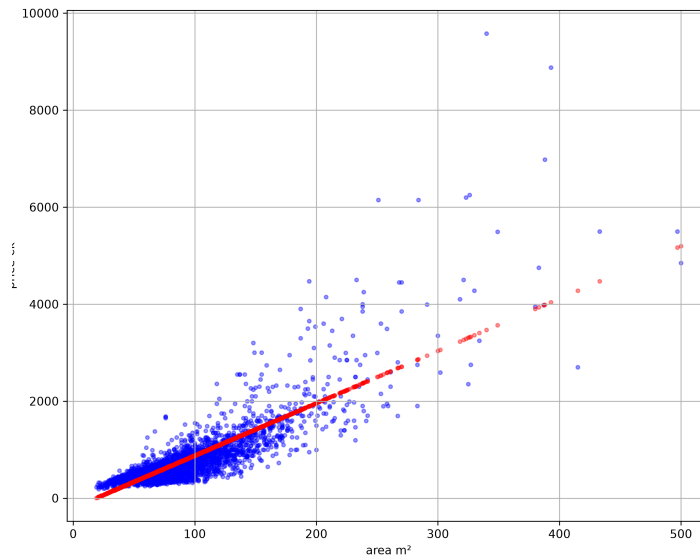


Figure 5: Model Results

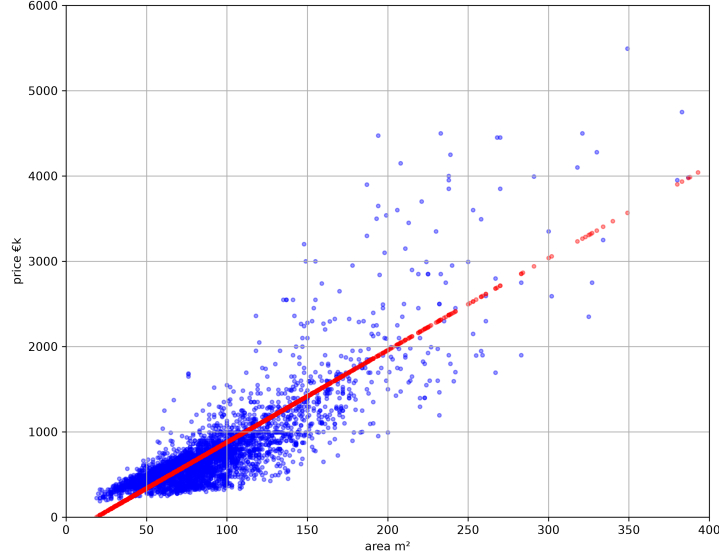


Figure 6: Model Results Without Outliers

This model has converged its weight and bias values to around 0.553 and -0.0197, respectively. This achieved the loss of 4.798×10^{-4} . I want to remind you that in previous example I achieved less loss, 0.81×10^{-4} because the loss was calculated across one entry which practically has a certain weight and bias that achieves 0 loss. $y = wx + b$ where y and x are constants means that there are infinitely many solutions.

On the other hand, here are 4500 entries that are impossible to cover with a linear equation like this. This line that my model has learned is just the best perfect line. Furthermore, the blue dots are arranged in a mostly chaotic way with a tendency of the price increasing as the area increases. This suggests that only area is not enough to fully predict price of house.

Nevertheless, this model can predict housing, although accuracy is limited. I can build a function according to the weight and bias I got:

$$y(x) = 0.5497 \times x - 0.0186$$

Let's test it on this example:

$$\text{Area} = 79 \text{ m}^2$$

$$\text{Target price} = 700 \text{ k } \text{€}$$

First normalize the data:

$$x = \frac{79 - 19}{500 - 19} = 0.1247$$

Use the function:

$$y(x) = 0.5497 \times x - 0.0186$$

$$\Rightarrow y(0.1247) = 0.5497 \times 0.1247 - 0.0186 = 0.0499$$

Now we can denormalize y by rearranging the normalization function:

$$y_{\text{norm}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}}$$

$$y_{\text{norm}} \times (y_{\max} - y_{\min}) = y - y_{\min} \tag{9}$$

$$y = y_{\text{norm}} \times (y_{\max} - y_{\min}) + y_{\min}$$

$$y = 0.0493 \times (9850 - 189) + 189 = 671.46$$

The target was 700k and the model predicted 671.46k which is not too bad for a model that only has the information about living area.

To assess how good this model actually is, this process must be done for more than one apartment. I put this model through my test dataset, with the results I got I can calculate the mean absolute error MAE and mean percentage absolute error MAPE.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - t_i| \quad (10)$$

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - t_i|}{t_i} \quad (11)$$

I got $\text{MAE} = 162.0k$ and $\text{MAPE} = 25.5\%$. Here are few other examples of predictions this model has made:

Target t (k €)	359.0	450.0	475.0	650.0	685.0	880.0
Predicted y (k €)	454.1	357.5	378.9	604.3	840.4	1205.3

$3.749\text{e-}04$ is the least loss I was able to get while using only one feature of the apartments.

2.6 Building the Model with 3 Inputs

Now, I can calculate the distance to the center of Amstedeam for each apartment. On my dataset, I have got the minimum distance 0.07km and the maximum 9.98km. So to normalize the distance I will use:

$$x_{\text{norm}} = \frac{x - 0.07}{9.98 - 0.07}$$

Using the data about the distance from the center of Amsterdam, I can add a new input variable for the model - x_3 . So, the function becomes:

$$y(x_1, x_2, x_3) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

From here I can get all the derivatives necessary, using derivatives from Equations 3, 4 and 6. As always, while deriviving partially, I treat other variables as constants:

$$\partial L / \partial w_1 = (y - t) * x_1$$

$$\partial L / \partial w_2 = (y - t) * x_2$$

$$\partial L / \partial w_3 = (y - t) * x_3$$

$$\partial L / \partial b = y - t$$

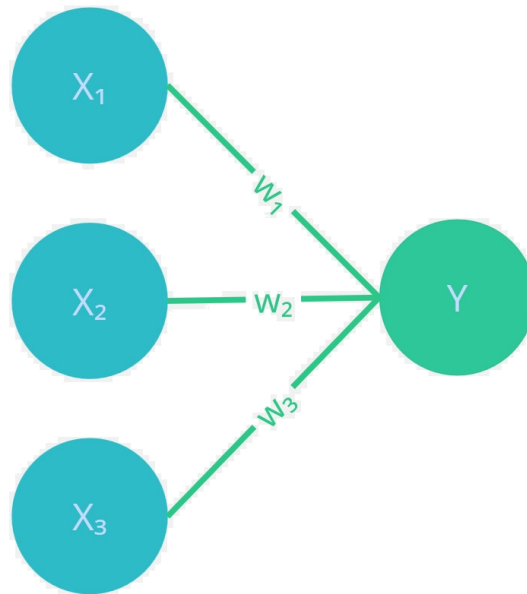


Figure 7: 3 Input Model

With this I can train a model for 1000 epochs. These are the weights and the bias, the model has converged to:

$$w_1 = 0.5429$$

$$w_2 = -0.0077$$

$$w_3 = -0.0534$$

$$b = 0.0072$$

Let's test it on the previous example:

$$\text{Area} = 79 \text{ m}^2$$

$$\text{Year} = 2005$$

$$\text{Distance} = 1.806 \text{ km}$$

$$\text{Target price} = 700 \text{ k } \text{€}$$

Normalize inputs:

$$x_1 = \frac{79 - 19}{500 - 19} = 0.1247$$

$$x_2 = \frac{2005 - 1507}{2028 - 1507} = 0.9559$$

$$x_3 = \frac{1.806 - 0.07}{9.98 - 0.07} = 0.1752$$

Calculate:

$$\begin{aligned}
y(0.1247, 0.9559, 0.1752) &= 0.5429 \times 0.1247 + \\
&+ -0.0077 \times 0.9559 + \\
&+ -0.0534 \times 0.1752 + \\
&+ 0.0072 = \\
&= 0.0582
\end{aligned}$$

Denormalize according to Equation 9:

$$y \times (9850 - 189) + 189 = 751.34$$

This time the model overshoot the price, but this model has achieved the best results with the test dataset on average making 3.01×10^{-4} loss, and having the MAE and MAPE of 130.2 and 18.3%, respectively.

Here are some other results:

Target t (k €)	359.0	450.0	475.0	650.0	685.0	880.0
Predicted y (k €)	317.0	467.3	354.6	587.3	890.9	1105.1

2.7 Influence of Distance on Price

Just like last time we can measure the influence of each input by the absolute values of their weights:

$$|w_1| = 0.5429$$

$$|w_2| = 0.0077$$

$$|w_3| = 0.0534$$

We can see that the most influential input is still area, but the importance

of construction year dramatically fell from 0.05 to 0.008, and the influence of distance is in middle of two other inputs.

Here is how the distribution of distance across the dataset looks like, (I have removed few outliers again):

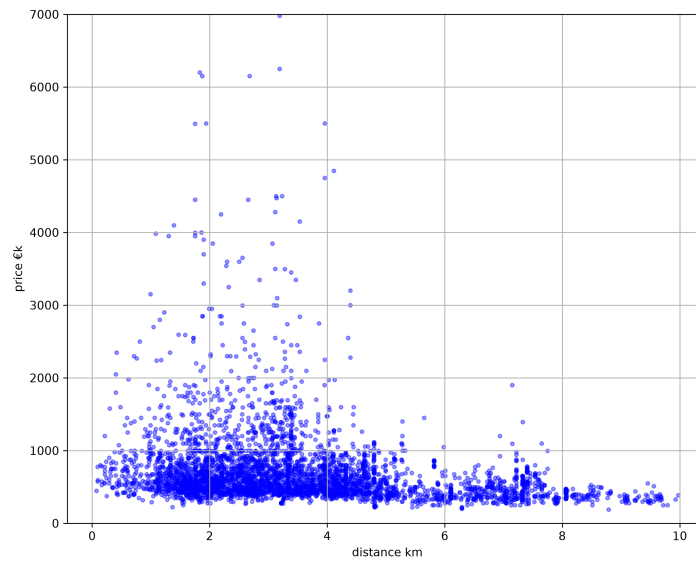


Figure 8: Distance against Price Data

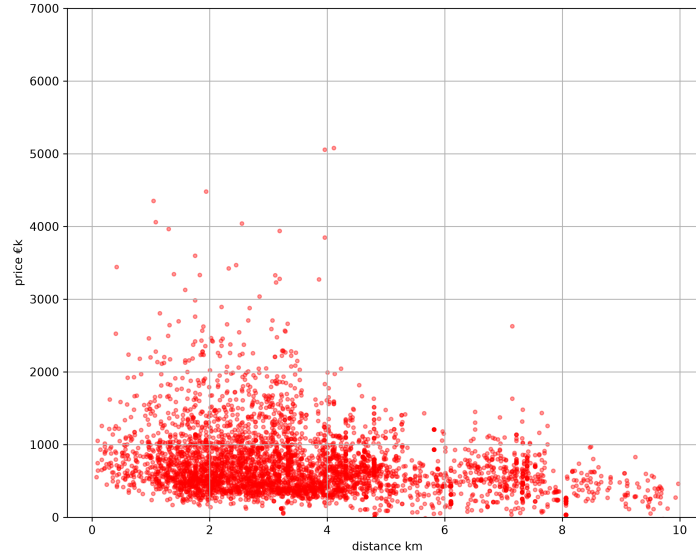


Figure 9: Distance against Price Predictions

The model was able to even capture the two spikes in price at distance 0 - 4km and 6 - 8km.

# of inputs	average loss	MAE	MAPE
1	3.7487×10^{-4}	162.0	25.5%
2	3.4844×10^{-4}	149.5	22.9%
3	3.0009×10^{-4}	130.2	18.3%